



universität
uulm

Interactive Computer Graphics

Chapter 2 – The Rendering Pipeline

Summer Term 2026

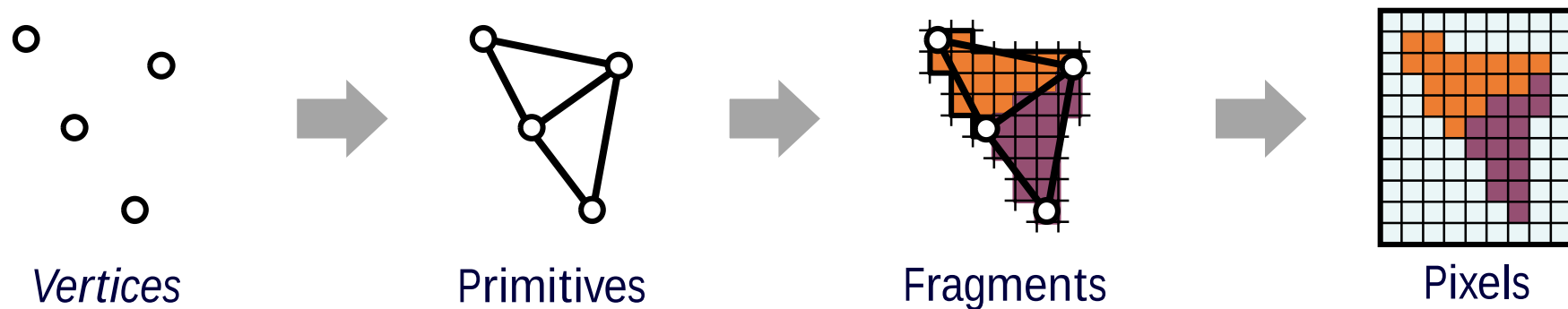
Timo Ropinski



Visual Computing Group
Institute of Media Informatics

Object-based Rendering

- Image generation done through rasterization
 - Scene objects formed by 3D models (primitives) are transformed through successive coordinate systems
 - Primitives are rasterized into fragments, which are pixel predecessors
 - A subset of the fragments becomes pixels



Chapter Outline

2.1 Rendering Process

2.2 OpenGL Overview

2.3 OpenGL Rendering

2.4 OpenGL Rendering Pipeline

2.5 Fragment Tests and Operations

2.6 Framebuffer

2.7 Pixel-based Rendering

References



universität
uulm

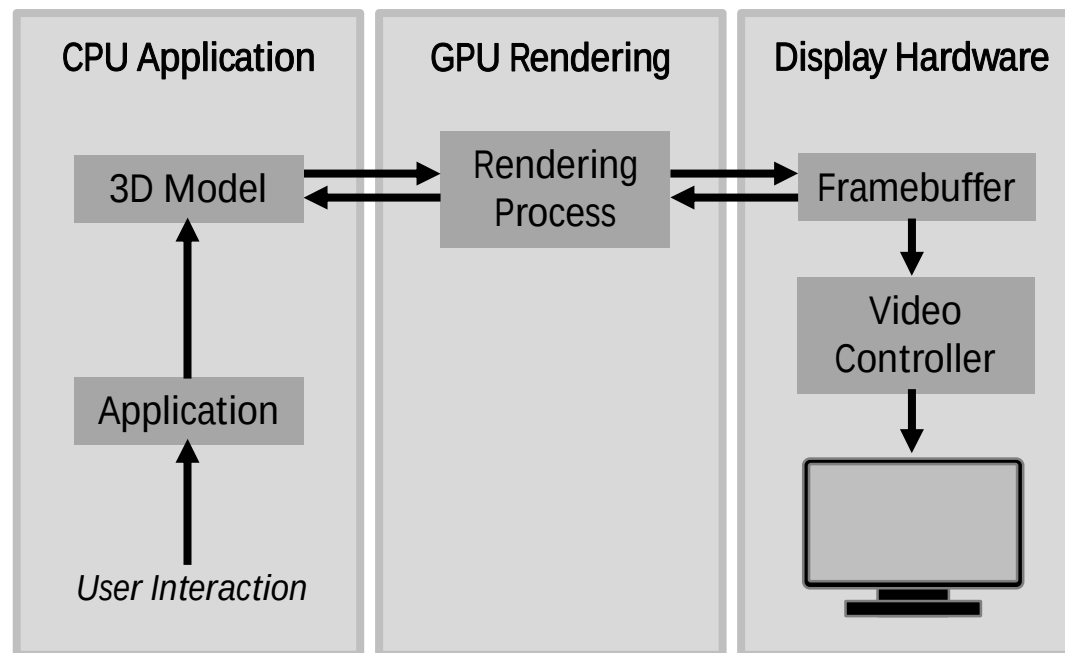
2.1 Rendering Process

From 3D models over fragments to pixels



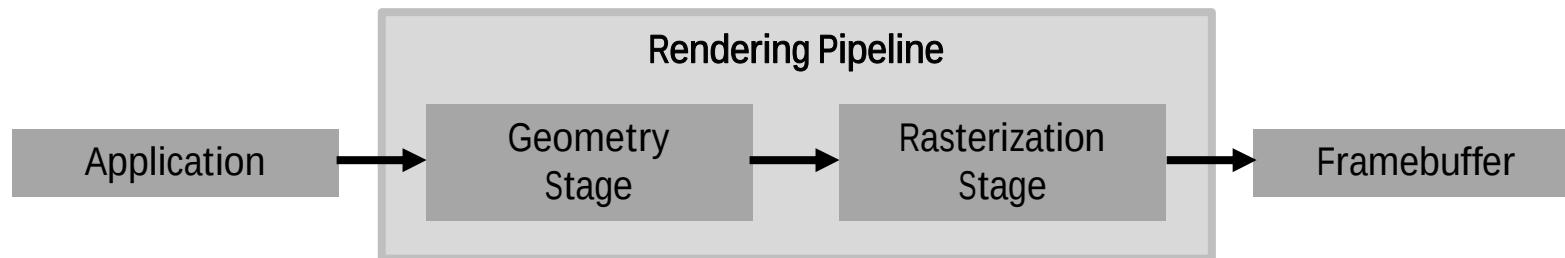
Visual Computing Group
Institute of Media Informatics

Geometry-based Rendering 1/2



Geometry-based Rendering 2/2

- The rendering process is realized through the rendering pipeline
 - Renders 3D models to 2D images
- Conceptually, the rendering pipeline consists of two GPU stages
 - Geometry stage – processes geometric primitives
 - Rasterization stage – processes fragments and pixels

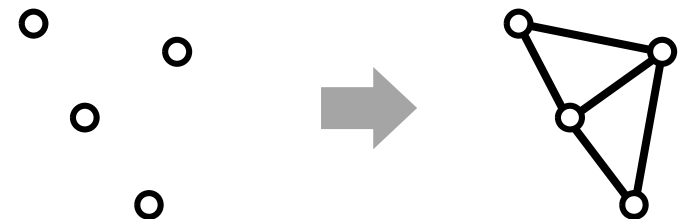
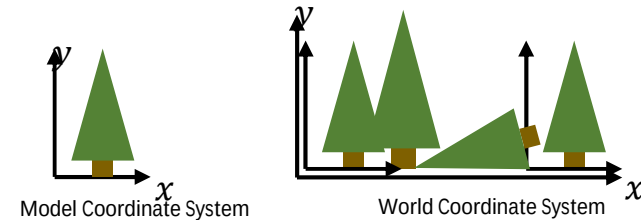


Application Stage

- Specification of the model elements and the scene as well as interaction capabilities
 - Scene modeling: arrangement and attribution of the scene objects
 - Calculation of animation paths
 - Collision detection
 - Selection of visible objects (occlusion culling)
- Mostly out of scope of this lecture course

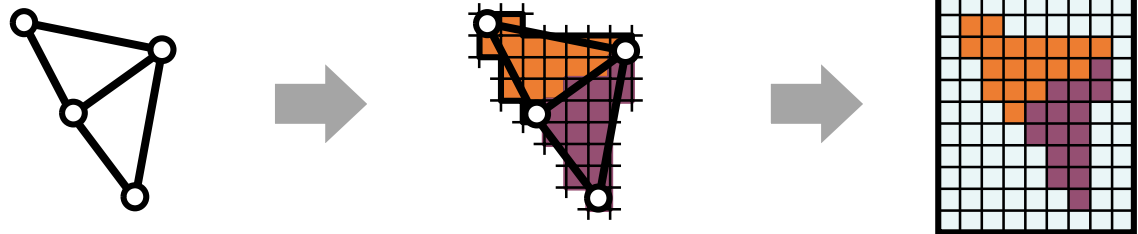
Geometry Stage

- Geometric 3D transformation
 - Alignment of individual scene objects in the overall scene
 - Basic transformations: translation, scaling, rotation
- Primitive assembly and tessellation
 - Composition of vertices to primitives
 - Generation of additional triangles via tessellation
- Geometric projection (projection)
 - 3D to 2D (e.g., perspective projection or orthogonal projection)
- Clipping
 - Removal of primitives outside of canvas
- Contains programmable substages



Rasterization Stage

- Conversion of primitives into fragments (rasterization)
 - Discretization takes place in relation to a target grid
 - Rasterization algorithms optimized for certain primitives (e.g., lines, triangles)
- After rasterization per-fragment tests are performed
 - Fragments undergo various tests (e.g., depth, transparency, masking)
 - Fragments are copied to the framebuffer if they pass all tests
 - Fragments can be blended with the current pixel values





universität
uulm

2.2 OpenGL Overview

Library for real-time computer graphics



Visual Computing Group
Institute of Media Informatics

OpenGL History

- Unified graphics software interface was missing as knowledge channeled into proprietary software packages
 - HP's starbase
 - SGI's Graphics Library (GL)
- Some standardization efforts existed
 - PHIGS
 - GKS
- Origin of OpenGL
 - SGI's GL paired with hardware
 - OpenGL specification driven by Mark Segal & Kurt Akeley



[<http://www.computinghistory.org.uk/det/11262/SGI-O2/>]

OpenGL

- OpenGL (Open Graphics Library)
 - Platform independent 2D and 3D graphics API
 - API specification is provided by the Khronos Group
- Technical implementation
 - Rendering pipeline consisting of static and programmable pipeline stages
 - Typically implemented as an ANSI C API (but with other bindings, such as Java)
 - Programmable pipeline stages are programmed in GLSL (OpenGL Shading Language)
- Implementation is hardware-dependent and therefore integrated in the graphics driver
- Meanwhile Khronos also released low-level API Vulkan



OpenGL vs. Vulkan

- How about Vulkan?
 - More modern than OpenGL
 - Designed for high performance and low-level GPU control
 - Better suited for complex, large-scale rendering engines
 - Vulkan also harder to learn and requires more setup/code
 - OpenGL is still great for learning the graphics basics first
 - Vulkan becomes important once we want more control and efficiency
- Takeaway: learn OpenGL for fundamentals, know Vulkan for current practice

OpenGL Libraries

- Different tastes of OpenGL
 - OpenGL
 - OpenGL ES - limited version for embedded systems
 - WebGL - Javascript connection for Web Development (functionality similar to OpenGL ES)
- GUI integration
 - Can be done through platform-specific integration (Windows: WGL, X11: GLX, Mac OS X: Cocoa)
 - Platform-independent solutions also exist (GLUT, freeGLUT, GLFW)
- Popular auxiliary libraries
 - GLU (OpenGL Utility Library) - collection of utility functions
 - GLEW (GL Extension Wrangler) - management of extensions
 - GLM library for vector and matrix operations

OpenGL Functionality

- Functional API
 - Objects, scenes and images are described by a sequence of commands (functional approach, low-level)
 - Contrast: scene or object description (declarative approach, high-level)
- OpenGL context
 - Sum of all rendering attributes with current characteristics
 - Acts as a state machine, whereby the current state can be changed by OpenGL function calls
 - Primitives are rendered depending on the current state

OpenGL Context

- Context is created and modified by the application, and contains all the state variables of the OpenGL engine
- Any number of contexts per application possible, but at most one active context per application
 - Context is generally tied to an output window
 - Requires management of the active OpenGL context
- Specification of global graphical attributes is independent of rendering primitives

OpenGL Rendering Process

- Initiation from the developer's perspective
 - Creation of the output window
 - Creation of the context
 - Specification of the scene geometry
 - Uploading of the scene geometry to the GPU
 - Rendering of the scene geometry

Window Creation with GLFW 1/2

```
#include <GLFW/glfw3.h>
#include <stdlib.h>
#include <stdio.h>

static void error_callback(int error, const char* description) {
    fputs(description, stderr);
}

static void key_callback(GLFWwindow* window, int key, int scancode, int action, int mods) {
    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
        glfwSetWindowShouldClose(window, GL_TRUE);
}
```

Window Creation with GLFW 2/2

```
int main(void){
    GLFWwindow* window;
    glfwSetErrorCallback(error_callback);
    if (!glfwInit()) exit(EXIT_FAILURE);
    window = glfwCreateWindow(640, 480, "Simple example", NULL, NULL);
    if (!window) {
        glfwTerminate();
        exit(EXIT_FAILURE);
    }
    glfwMakeContextCurrent(window);
    glfwSetKeyCallback(window, key_callback);
    while (!glfwWindowShouldClose(window)) {
        renderScene();
        glfwSwapBuffers(window);
        glfwPollEvents();
    }
    glfwDestroyWindow(window);
    glfwTerminate();
    exit(EXIT_SUCCESS);
}
```

OpenGL Naming Conventions

- Function-Prefix "gl"
 - `glClear`, ...
- Constant-Prefix "GL"
 - `GL_POLYGON`, `GL_LINES`, ...
- Parameter types are explicitly encoded in function names

Suffix	Data Type	C-Data Type	OpenGL typedef
b	8-bit Integer	signed char	GLbyte
s	16-bit Integer	short	GLshort
i	32-bit Integer	int / long	GLint, GLsizei
f	32-bit Floating-Point	float	GLfloat, GLclampf
d	64-bit Floating-Point	double	GLdouble, GLclampd
ub	8-bit unsigned Integer	unsigned char	GLubyte, GLboolean
us	16-bit unsigned Integer	unsigned short	GLushort
ui	32-bit unsigned Integer	unsigned int / long	GLuint, GLenum, GLbitfield
v	Pointer	*	-



universität
uulm

2.3 OpenGL Rendering

Transfer of 3D models to the GPU

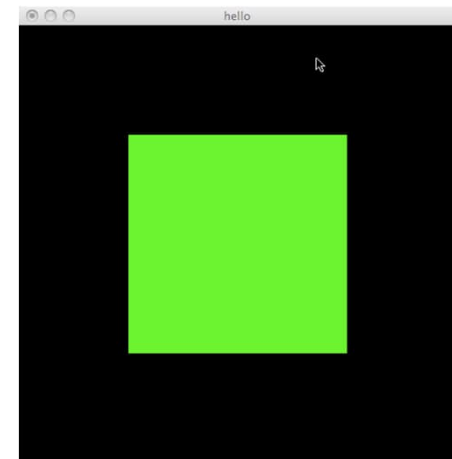


Visual Computing Group
Institute of Media Informatics

OpenGL Rendering (until V2.1)

- Rendering is done through Immediate Mode

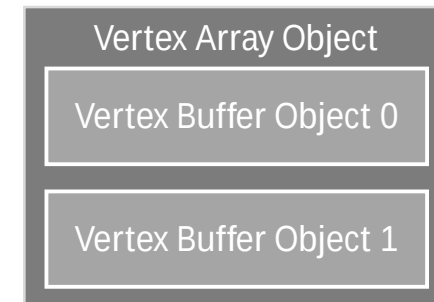
```
void renderScene() {  
    glColor3f(0.0f, 1.0f, 0.0f); // drawing color becomes green  
    glBegin(GL_POLYGON);          // begin primitive  
        glVertex2f(-0.5f, -0.5f); // 1. vertex  
        glVertex2f(-0.5f, 0.5f);  // 2. vertex  
        glVertex2f( 0.5f, 0.5f);  // 3. vertex  
        glVertex2f( 0.5f, -0.5f); // 4. vertex  
    glEnd();                      // end primitive  
    glFlush();                    // synchronize pipeline  
}
```



- Disadvantages
 - Transfer geometry from the CPU to the GPU on every call
 - Transmission takes much longer than the actual drawing process

OpenGL Rendering (since V3.0)

- Using Vertex Array Objects (VAOs)
 - Vertex specification in CPU memory
 - Vertex data transfer from CPU to GPU
 - Memory layout description of the vertex data
 - Number of vertices, number of components (x, y, z, ...)
 - Base type (float, double, ...)
 - ...
 - VAO Drawing
 - GPU memory release
- VAO acts as a container for the geometry data stored in one or more Vertex Buffer Objects (VBOs)



1. Vertex Specification

- Vertex data is specified as a C array

```
// vertices of two triangles
GLfloat vertices[6][2] = { // 6 vertices with two components each (x,y)
    {0.0, 1.0}, {1.0, 1.0}, {1.0, 0.0}, // 1. triangle
    {0.5, 0.5}, {1.0, 0.3}, {0.7, 1.0} // 2. triangle
};
```

- In OpenGL an ID must be generated for each VAO and VBO
- Active VAO and VBO are selected by ID

```
GLuint vaoID;
glGenVertexArrays(1, &vaoID);
glBindVertexArray(vaoID);
GLuint vboID;
glGenBuffers(1, &vboID);
glBindBuffer(vboID);
```

2. Vertex Data Transfer

- VBO allocation and data transfer via
- `glBufferData(GL_ARRAY_BUFFER, size, data, GL_STATIC_DRAW);`
 - `data` – address in main memory (i.e., `vertices` or `&vertices[0]`)
 - `size` – size of data in number of bytes
(i.e. `sizeof(vertices)` or `numVertices*sizeof(GLfloat)*2`)
 - `GL_STATIC_DRAW`: Buffer is generated once and used several times
(alternative: `GL_DYNAMIC_DRAW`)
- In the case of `data == NULL`, only the memory on the GPU is reserved, but there is no data transfer

3. Memory Layout Description

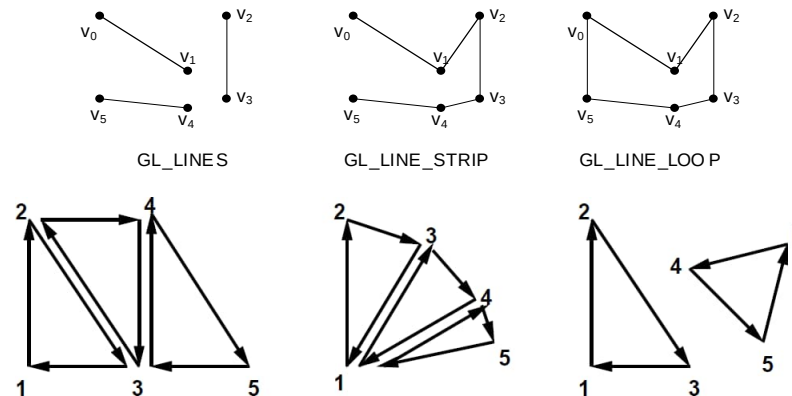
- Memory layout is described by the following function
- `glVertexAttribPointer(attribIndex, size, type, normalized, stride, offset);`
 - **attribIndex** – attribute number (0 for position, later more ...)
 - **size** – number of components per vertex (2 for xy, 3 for xyz)
 - **type** – base type (typically `GL_FLOAT` or `GL_DOUBLE`)
 - **normalized** – normalize integer values when converting to floating-point numbers to `[-1,1]` (signed) or `[0,1]` (unsigned)
 - **stride** – gap between data in memory (0: data directly adjacent to each other)
 - **offset** – is added to the start address specified with `glBufferData` (can often be simply set to 0)
- First, attributes must be activated
- `glEnableVertexAttribArray(attribIndex);`
- The coordinates of the vertices are assigned to the attribute index 0, but there is also the possibility to assign further attributes (e.g., colors, normals, ...)

4. VAO Drawing 1/2

- Drawing is done with the command
- `glDrawArrays(mode, first, count);`
 - `mode` – specifies the primitive type (e.g., `GL_TRIANGLES` or `GL_LINES`)
- From all vertices, the next `count` vertices are drawn from index `first`
- Actual vertex data no longer needs to be specified

4. VAO Drawing 2/2

- There is a small number of primitive types
 - Points
 - Lines, Line Strips, Line Loops
 - Triangles, Triangle Strips, Triangle Fans, Patches



- For Geometry shaders: with adjacency (discussed later)

5. GPU Memory Release

- After usage VAO and VBO should be deleted from GPU memory
- `glDeleteBuffers(...);`
- `glDeleteVertexArrays(...);`

Rendering Initialization 1/2

```
static void init_render_scene(void) {
    static const GLfloat vertices[6][2] = {
        {0.0f, 1.0f}, {1.0f, 1.0f}, {1.0f, 0.0f}, /* 1st triangle */
        {0.5f, 0.5f}, {1.0f, 0.3f}, {0.7f, 1.0f} /* 2nd triangle */
    };

    static const char* vs_src =
        "#version 330 core\n"
        "layout (location = 0) in vec2 aPos;\n"
        "void main(void) {\n"
        "    gl_Position = vec4(aPos, 0.0, 1.0);\n"
        "}\n";

    static const char* fs_src =
        "#version 330 core\n"
        "out vec4 FragColor;\n"
        "void main(void) {\n"
        "    FragColor = vec4(1.0, 1.0, 1.0, 1.0);\n"
        "}\n";
    ...
}
```

Rendering Initialization 2/2

```
program = create_program(vs_src, fs_src);

glGenVertexArrays(1, &vao);
glGenBuffers(1, &vbo);

glBindVertexArray(vao);

glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(GLfloat), (void*)0);
glEnableVertexAttribArray(0);

glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
}
```

Rendering

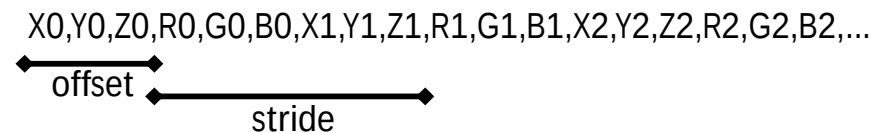
```
static void render_scene(void) {
    glClear(GL_COLOR_BUFFER_BIT);

    glUseProgram(program);
    glBindVertexArray(vao);
    glDrawArrays(GL_TRIANGLES, 0, 6);
    glBindVertexArray(0);
}

static void cleanup_render_scene(void) {
    if (vbo)    glDeleteBuffers(1, &vbo);
    if (vao)    glDeleteVertexArrays(1, &vao);
    if (program) glDeleteProgram(program);
}
```

Additional Attributes

- Additional attributes are necessary for drawing
 - Color, normals, texture coordinates, ...
- There are two ways to assign these attributes to the vertex data
 - Attributes are bound to the same VAO in separate VBO
 - Attributes are stored interleaved with vertex data, whereby stride and offset are used to separate





universität
uulm

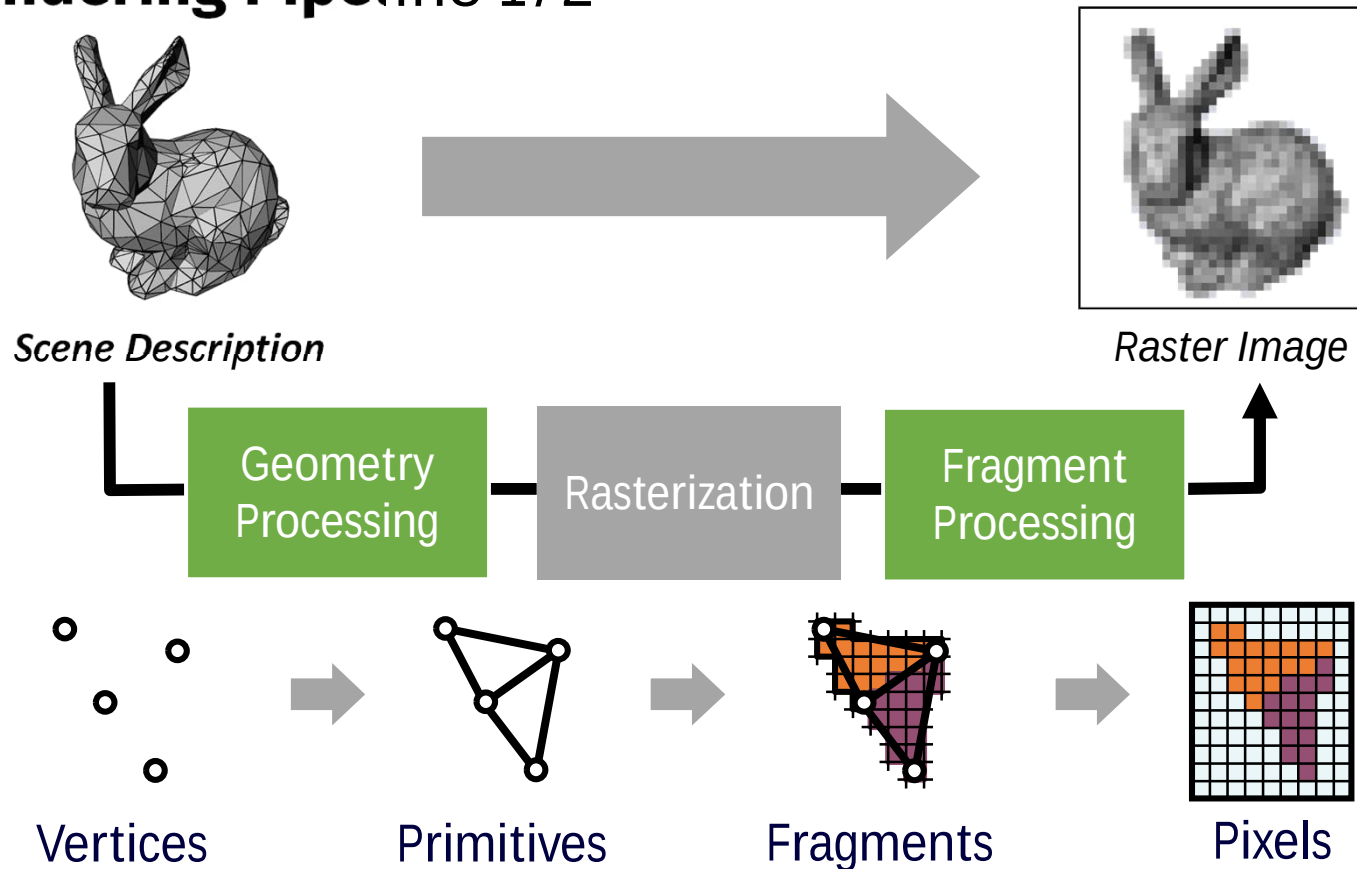
2.4 OpenGL Rendering Pipeline

Programmable and static stages



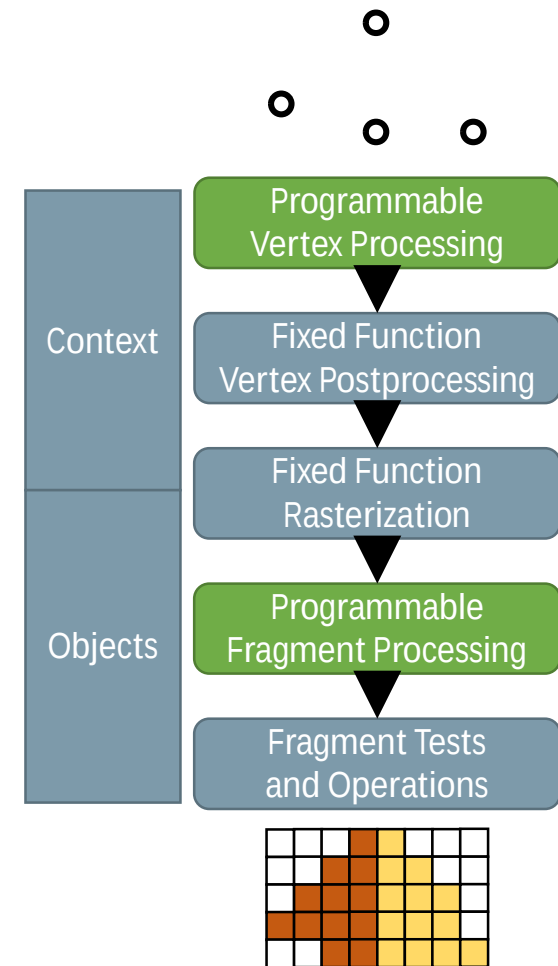
Visual Computing Group
Institute of Media Informatics

The Rendering Pipeline 1/2



The Rendering Pipeline 2/2

- Sequence of fixed and programmable stages
- Process is influenced by context and scene objects
- Programmable levels are programmed via shaders in GLSL



Shader Programming – History 1/2

- There is *special* graphics hardware (dedicated) and *general* graphics hardware (general purpose)
- Special graphics hardware
 - Volume Rendering: VolumePro
 - Game consoles: Custom NVIDIA chips in Xbox
 - Graphics chips for mobile devices
- General graphics hardware
 - SGI's InfiniteReality
 - UNC's PixelFlow
 - NVIDIA's GeForce and AMD's Radeon series

Shader Programming – History 2/2

- Various vendor-dependent extensions are introduced to provide extensibility for general graphics hardware
 - Texture shaders and register combiners allowed modification through the application of textures
- Applicability is very limited by the possibilities of graphics hardware
- General programming language for developing shaders was needed

Assembler Shader

- First OpenGL extensions support shader programming in assembler code

```
!! ARBvp1.0
ATTRIB iPos = vertex.position ;
ATTRIB iNormal = vertex.normal ;
...
OUTPUT oPos = result.position ;
OUTPUT oColor = result.color ;
# Transform the vertex to clip coordinates
DP4 oPos.x, mvp [0], iPos ;
DP4 oPos.y, mvp [1], iPos ;
DP4 oPos.z, mvp [2], iPos ;
DP4 oPos.w, mvp [3], iPos ;
# Assign color contributions
...
MOV oColor.w, diffuseCol.w;
END
```

High-Level Shading Languages 1/2

- Assembler code is hard to read and hard to maintain
- High-level languages for shading are specified in C-like syntax
- Source code is translated by the compiler into appropriate ASM statements
- Using shaders after initial binding automatically when rendering geometry
- Interface to OpenGL by passing appropriate variables or changing the OpenGL state (e.g., binding textures)

High-Level Shading Languages 2/2

- Existing high level languages for shader development
 - CG - C for Graphics by NVIDIA
 - CG was introduced in 2003 by NVIDIA as a platform independent solution (in practice rather semi-platform independent)
 - HLSL - High Level Shading Language from Microsoft
 - Microsoft introduced HLSL as an extension to DirectX in 2003 (fragment shaders are called pixel shaders)
 - GLSL - OpenGL Shading Language
 - GLSL has been supported since 2005 (initially as an OpenGL extension)
- GLSL syntax similar to C
 - Vector and matrix base types exist

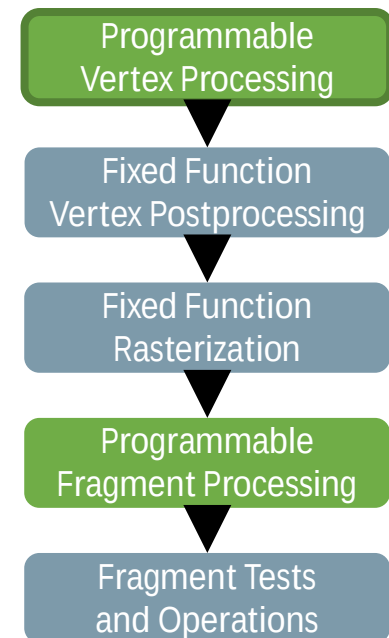
```
vec4 v1;    // same as 'float v1[4]' in C
ivec3 v2;   // same as 'int v2[3]' in C
vec4 v3 = v1.xzzy; // swizzling of components
```

Shader Types

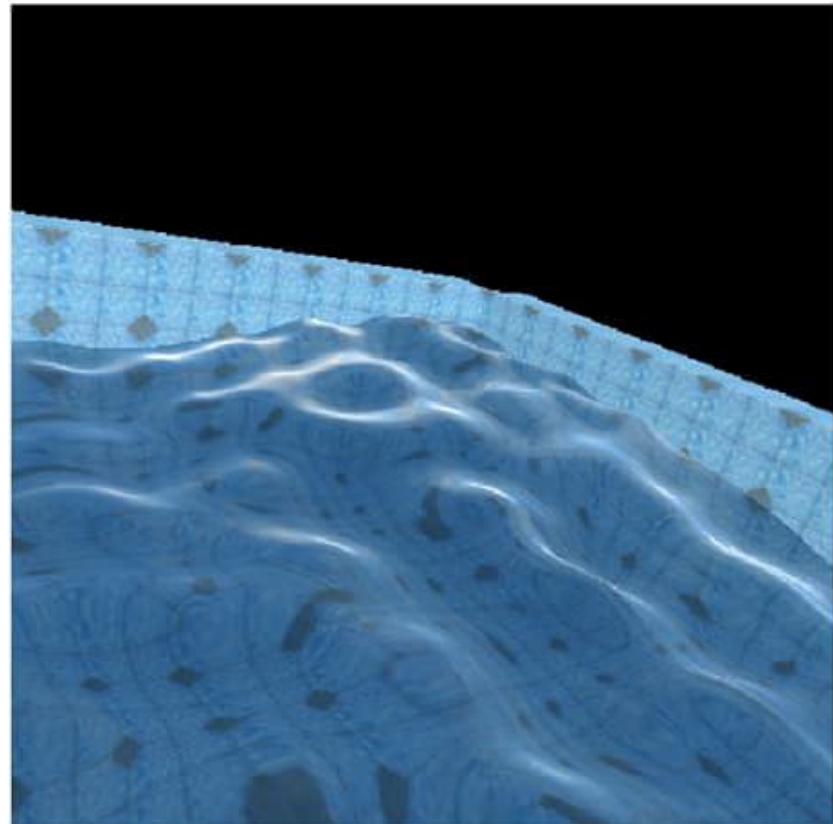
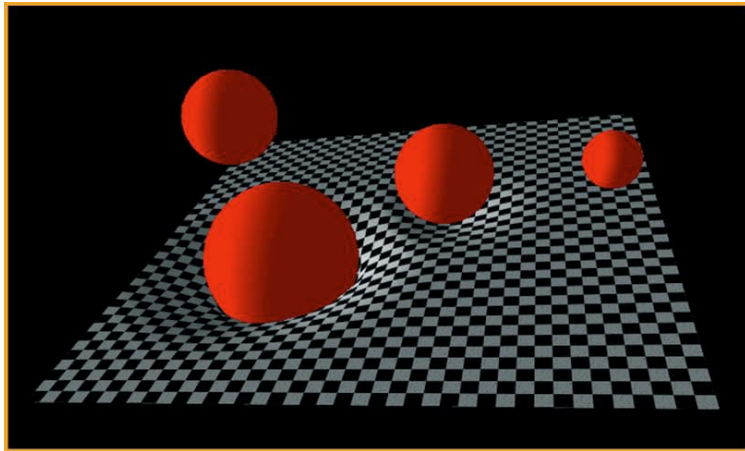
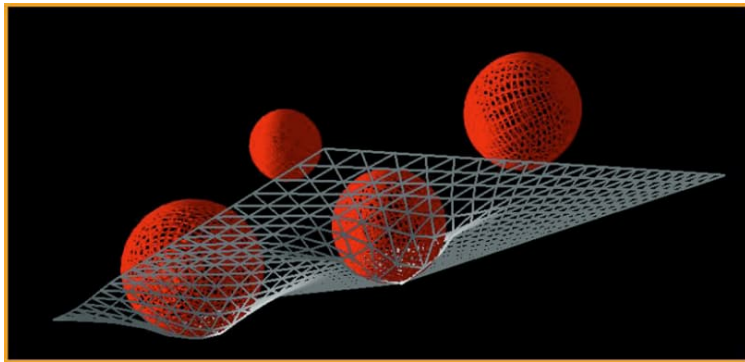
- Pipeline concept has been kept in place and shaders are responsible for different stages
- First, geometry level programming was done by vertex shaders
 - Vertex shaders allow modifiability of vertex transformations and lighting
- After that, there was an extension for image-based algorithms by fragment shaders
 - Fragment shaders allow programmability of texture applications
- This was followed by geometry shaders
 - Generation of new geometry (later chapter)
- Afterwards tessellation shaders were introduced that influence the triangulation of primitives

Vertex Shader Concepts

- Vertex shaders enable the programmable modification of processed geometry
- Possibilities of a vertex shader
 - Geometric transformation of vertices and normals
 - Lighting calculation
 - Texture coordinate generation
 - Texture coordinate transformation
 - Normalization of normals
 - Custom clipping planes
 - ...

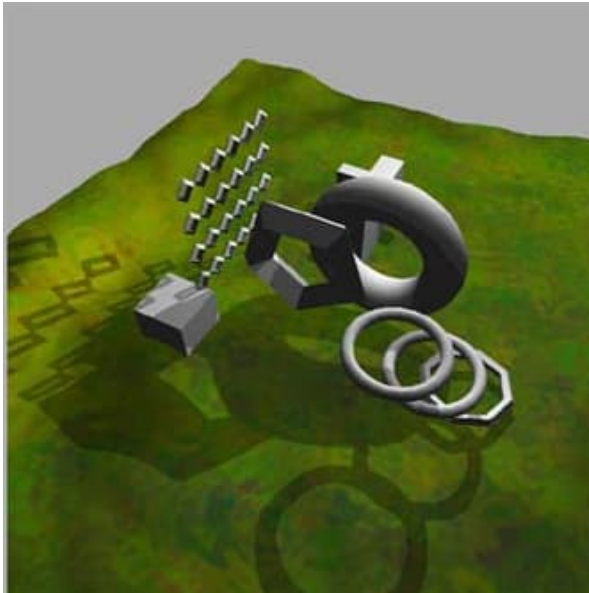


Vertex Shader Examples 1/2



Vertex Shader Examples 2/2

- Morphing
- Shadow volume calculation for shadow effects



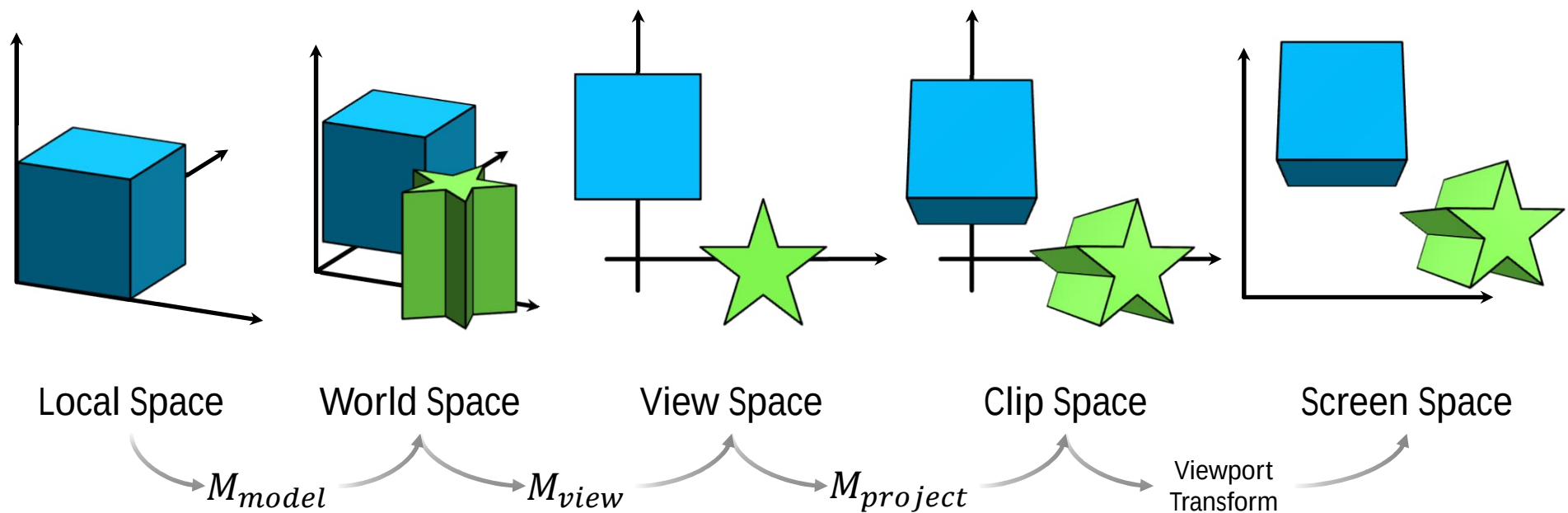
[ID Software's Doom III]

Vertex Shader Coordinate Systems 1/2

- Vertex shaders are executed for each vertex
- Vertex shaders calculate coordinates in clipping coordinates as output
- Vertex shaders can not
 - Access adjacent vertices
 - Change the number of vertices (i.e., generate or discard vertices)
- Output is always the final coordinate of the vertex

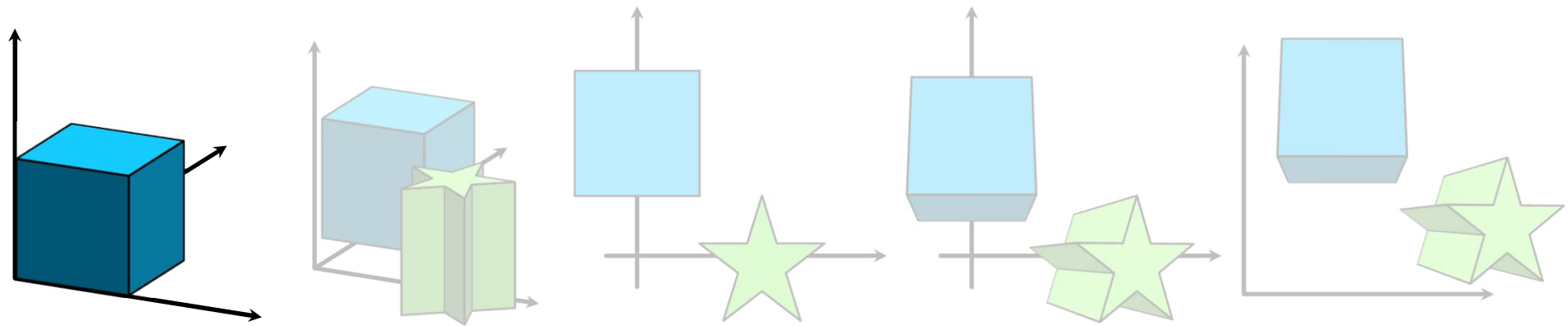
```
gl_Position = vec4(pos.x, pos.y, pos.z, pos.w);
```

Vertex Shader Coordinate Systems 2/2



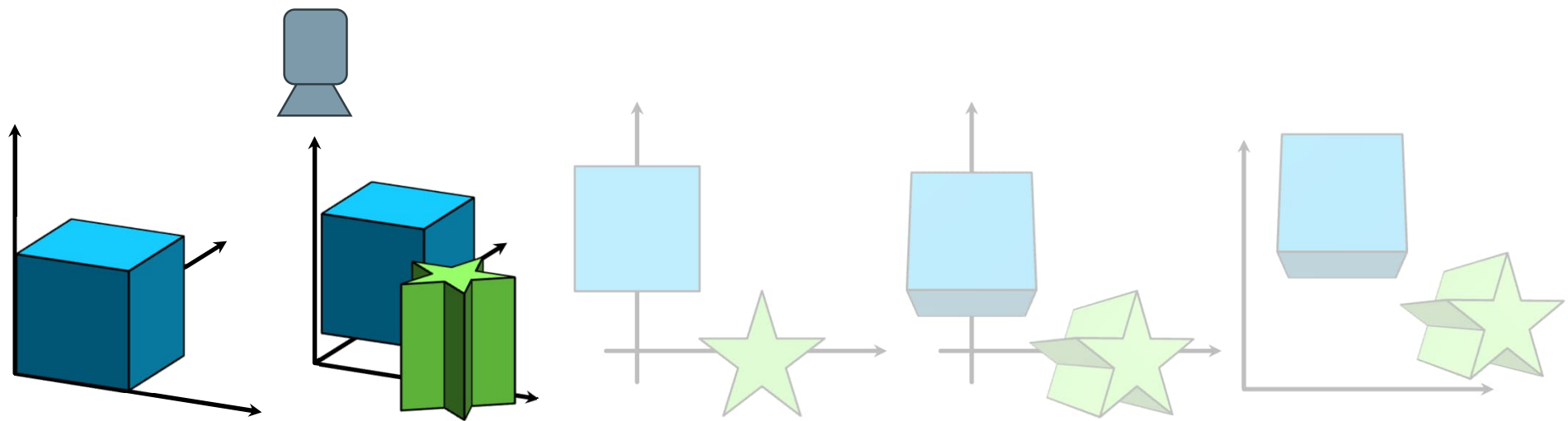
Local Space

- Coordinates of your object relative to its local origin
 - 3D vertex positions



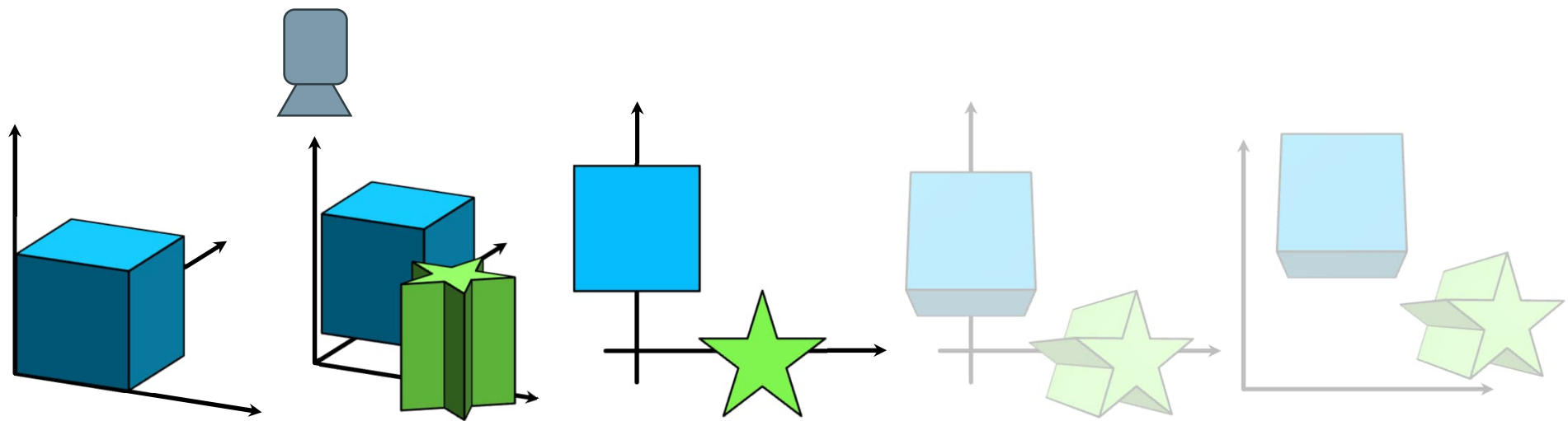
World Space

- Coordinates in respect of a larger world
- Relative to some global origin of the world
- Together with many other objects



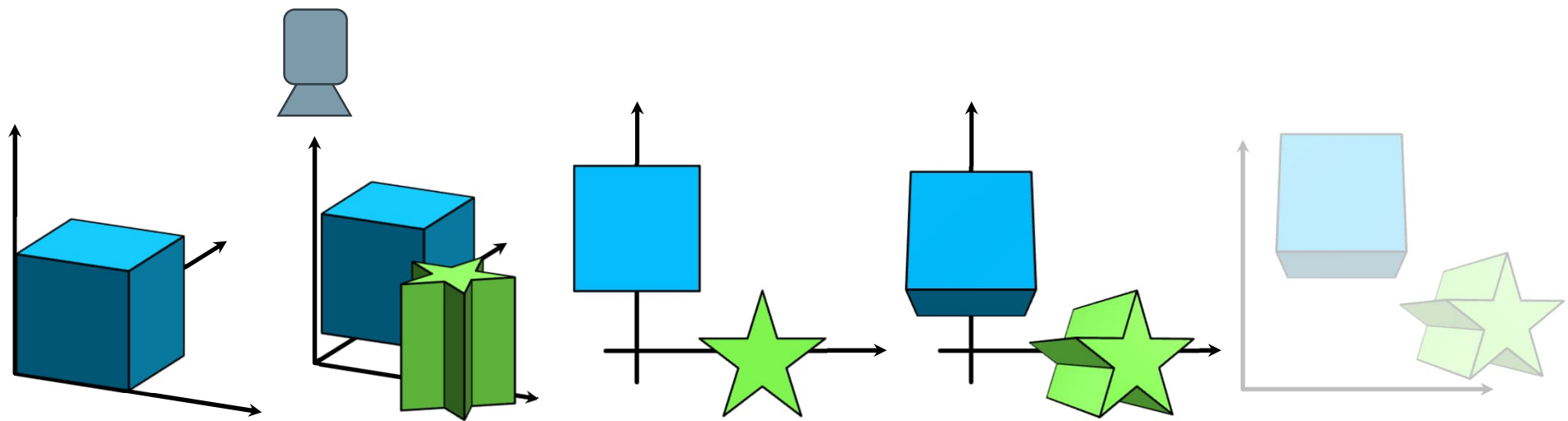
View Space

- Coordinates as seen from the camera



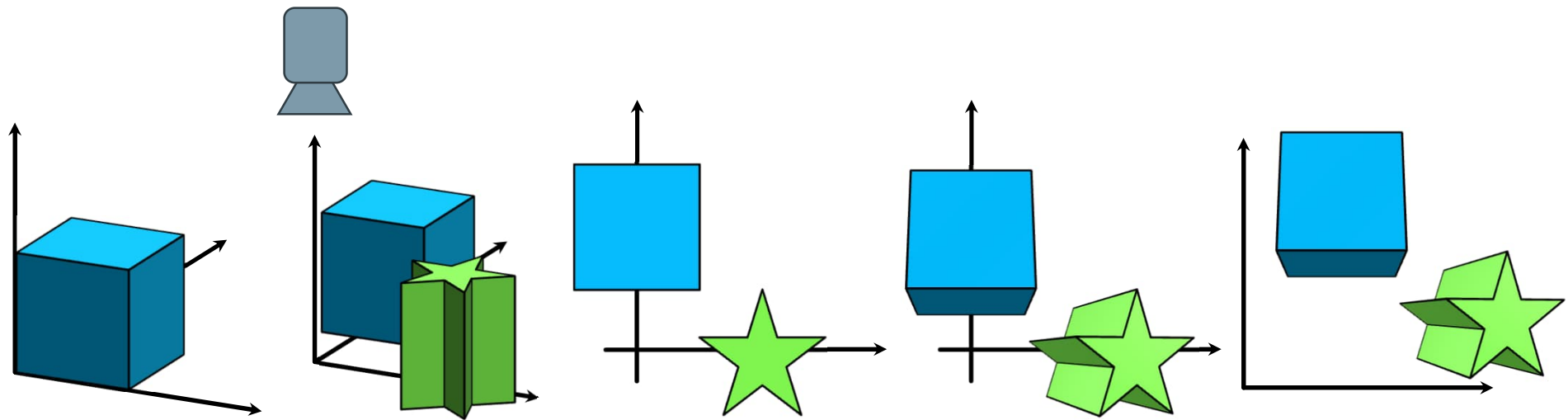
Clip Space

- Clip coordinates range from -1 to 1
- Projection (Orthographic, Perspective)

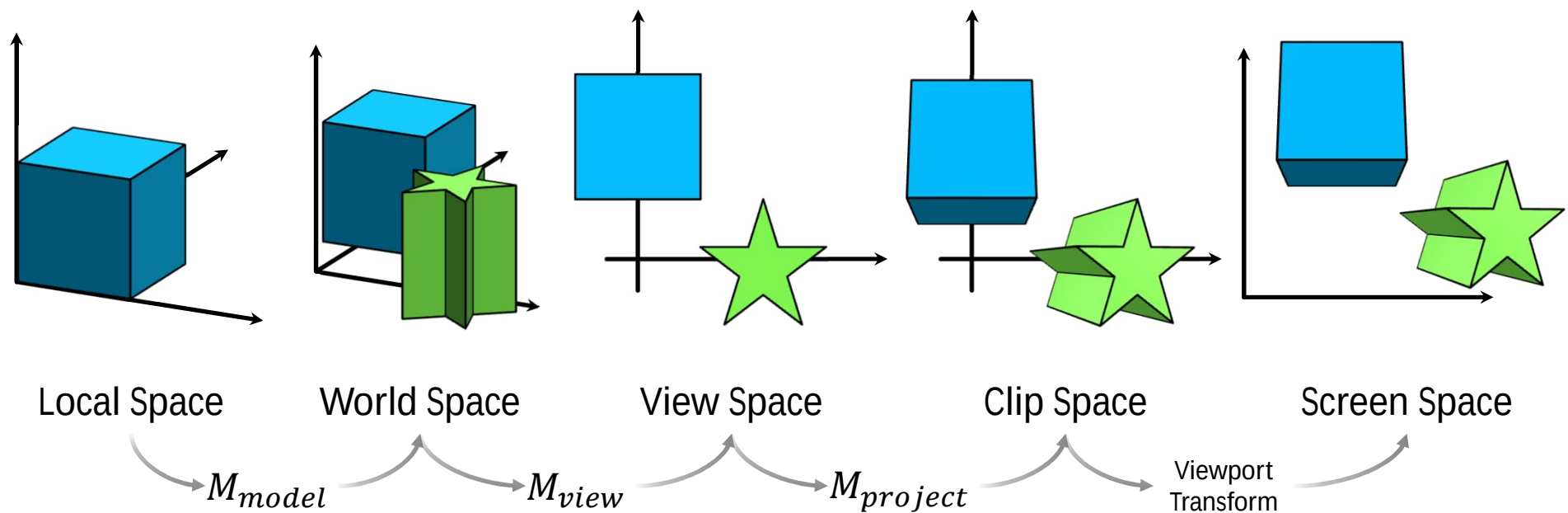


Screen Space

- Transform to image coordinates with `glViewport`
- Results are sent to the rasterizer



Coordinate Systems



Vertex Shader Variables

■ Input types

- Uniforms: read only in VS (& FS)
 - For example: light position or light color
- Attributes: Only accessible in the VS
 - For example: vertex position or normal
- Varyings: Data transfer from the VS to the FS
 - Read access in FS, read and write access in VS

■ Output types

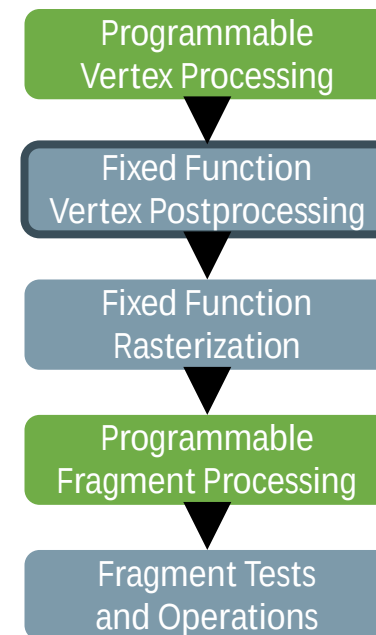
- Attributes for each vertex can be written

```
// Intrinsic vertex attributes
out gl_PerVertex {
    vec4    gl_Position;
    float   gl_PointSize;
    float   gl_ClipDistance[];
};
```

```
// Custom vertex attributes
out VertexData {
    vec2 texCoord;
} VertexOut;
...
VertexOut.texCoord = vec2(...);
```

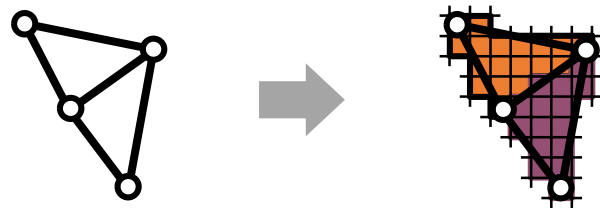
Fixed Function Vertex Postprocessing

- Clipping
 - Discard invisible geometries
- Perspective Division
 - Transformation into normalized device coordinates (NDC)
- Viewport Transformation
 - Transformation into screen coordinates
 - Scaling of depth values (see later)



Fixed Function Rasterization

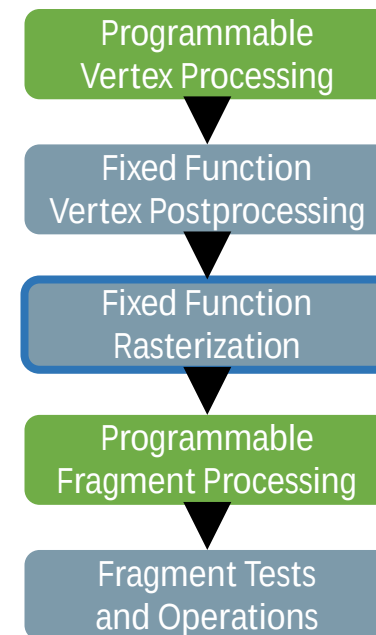
- Geometry is converted to raster coordinates



Primitives

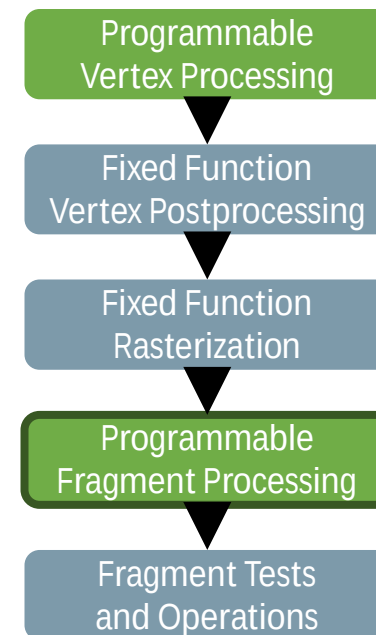
Fragments

- Efficient algorithms necessary
- Specializes in algorithms for different types of primitives
 - Line, triangle, ...



Fragment Shader Concept 1/2

- Fragment shaders set the color of a pixel
- Possibilities of a fragment shader
 - Glare of textures
 - Fog calculation
 - Discarding fragments
 - Additional depth tests
 - Additional alpha tests
 - Bump and environment mapping
 - ...



Fragment Shader Concept 2/2

- Fragment shaders are executed for fragments (= pixel predecessors)
- Fragment shaders calculate the color and depth value of the current fragment
- No information about adjacent fragments available
- The number of fragments can be changed
 - Fragment shaders can not generate fragments
 - Fragment shaders can discard fragments
- Output is always the final color of the fragment along with the depth

```
FragData0 = vec4(1.0, 0.0, 0.0, 1.0);  
gl_FragDepth = 0.5;
```

Shader Example

```
layout (std140) uniform Matrices {  
    mat4 projModelViewMatrix;  
    mat3 normalMatrix;  
};  
  
in vec3 position;  
in vec3 normal;  
in vec2 texCoord;  
  
out VertexData {  
    vec2 texCoord;  
    vec3 normal;  
} VertexOut;  
  
void main() {  
    VertexOut.texCoord = texCoord;  
    VertexOut.normal = normalize(normalMatrix * normal);  
    gl_Position = projModelViewMatrix * vec4(position, 1.0);  
}
```

Vertex Shader

```
/**  
 * Set fragment color to red.  
 */  
  
out vec4 colorOut;  
  
void main() {  
    colorOut = vec4(1.0, // R  
                    0.0, // G  
                    0.0, // B  
                    1.0); // A  
    gl_FragDepth = 0.5;  
}
```

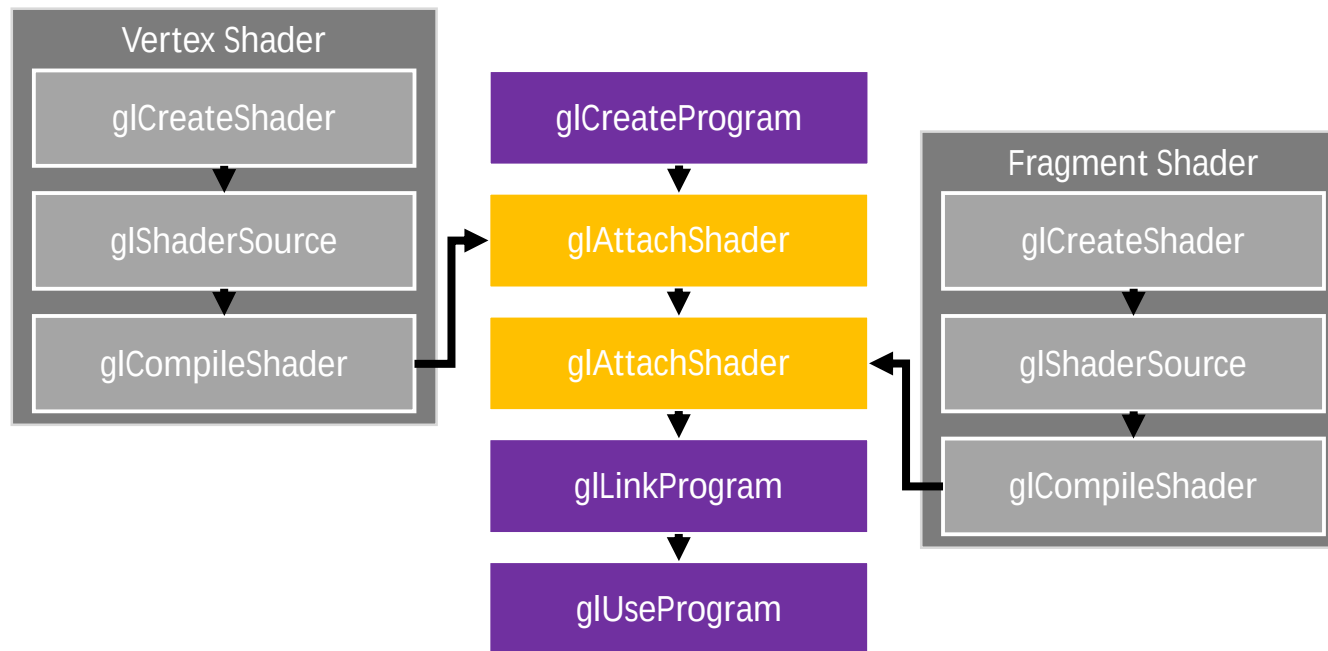
Fragment Shader

GLSL Shader Functions

- Trigonometry functions
 - radians, degrees, sin, cos, tan, asin, acos, atan
- Power calculations
 - pow, exp2, log2, sqrt, inversesqrt
- General
 - abs, sign, floor, ceil, fract, mod, min, max, clamp, mix, step, smoothstep
- Vector and matrix operations
 - length, distance, dot, cross, normalize, ftransform, faceforward, reflect, matrixCompMult
- And many more!

Shader Usage

- One or multiple shaders are attached to a shader program



Binding Shaders

```
v = glCreateShader(GL_VERTEX_SHADER); // vertex shader
char* vs = readTextFile("vertex-shader.vert");
const char* vv = vs;
glShaderSource(v, 1, &vv, NULL);
free(vs);
glCompileShader(v);

f = glCreateShader(GL_FRAGMENT_SHADER); // fragment shader
char* fs = readTextFile("fragment-shader.frag");
const char* ff = fs;
glShaderSource(f, 1, &ff, NULL);
free(fs);
glCompileShader(f);

p = glCreateProgram(); // create shader program
glAttachShader(p, v);
glAttachShader(p, f);
glLinkProgram(p);
glUseProgram(p);
```

Setting Uniforms

- Uniforms are used to communicate between CPU and GPU
- Data is bound to uniform locations
 - Must be updated after each linking process

```
GLint loc0 = glGetUniformLocation(p, "isoValue_");
glUniform1f(loc0, 0.45f);

float intensities[2] = {0.33f, 0.51f};
GLint loc1 = glGetUniformLocation(p, "intensities_");
glUniform1fv(loc1, 2, intensities);

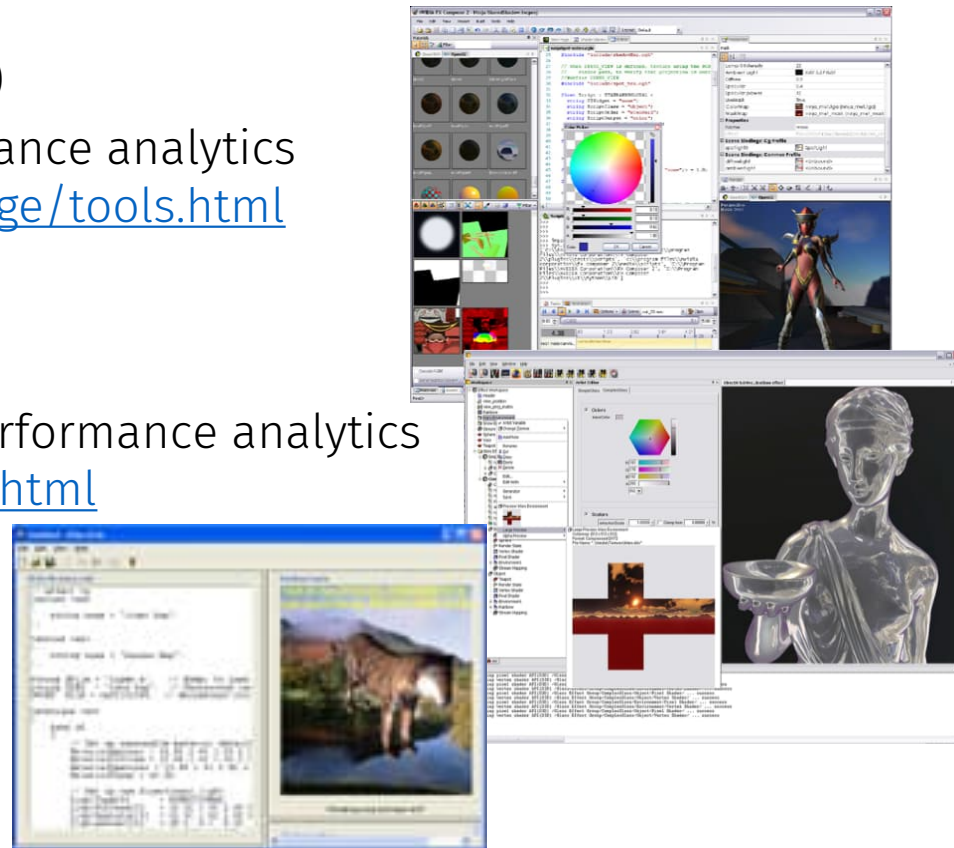
double vertices[12] = {0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, 0.0};
GLint loc2 = glGetUniformLocation(p, "vertices_");
glUniform3dv(loc2, 4, vertices);
```

Shader Performance

- Guidelines for efficient shaders
 - Since fragment shaders are executed for a large number of fragments, they should contain as few instructions as possible
 - There should be as little as possible a change of the active shaders, because they have their own state and, accordingly, the pipeline may need to be emptied
 - With frequent changes, the behavior of a shader should be changed via uniforms instead of replacing the shader

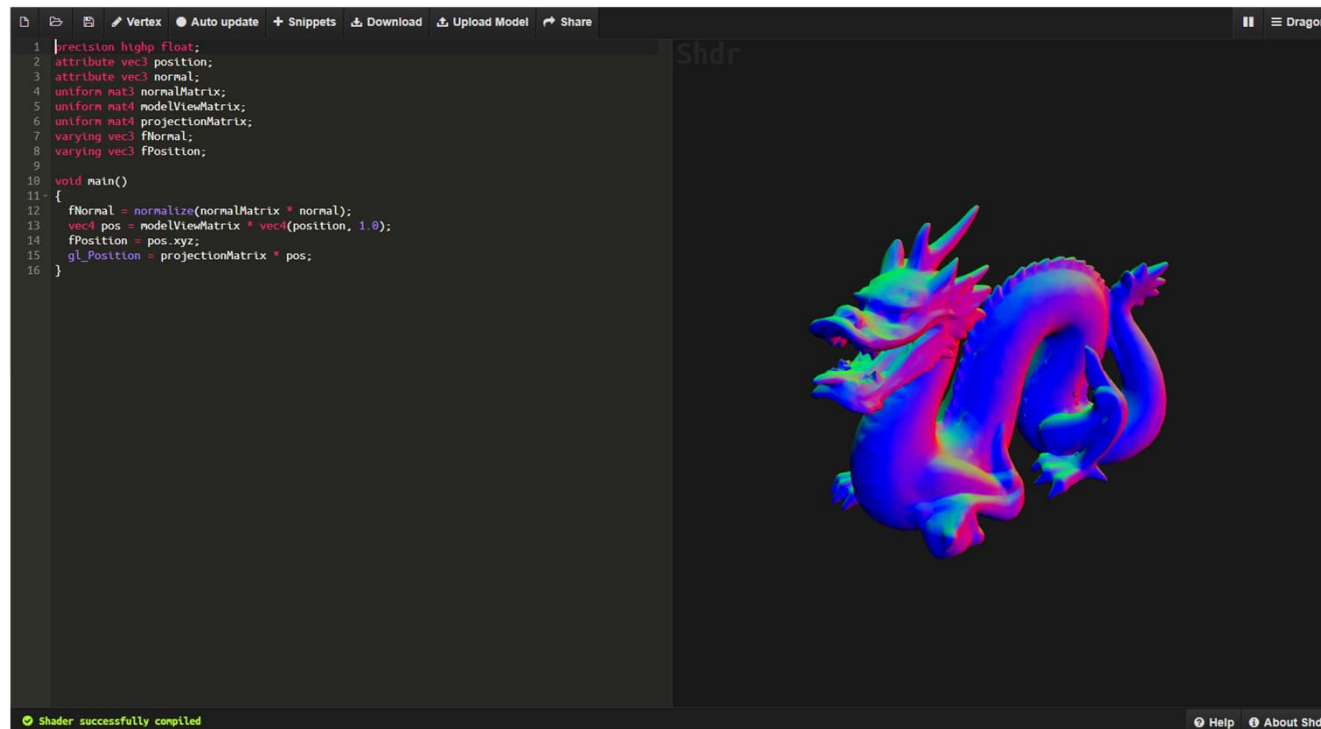
Shader Development Environments 1/2

- FX Composer, NVPerfHUD (NVIDIA)
 - HLSL Shader IDE with performance analytics
www.developer.nvidia.com/page/tools.html
- RenderMonkey (ATI)
 - HLSL, GLSL Shader IDE with performance analytics
www.ati.com/developer/tools.html
- EffectEdit (Microsoft)
 - Interactive HLSL Renderer
<http://msdn.microsoft.com/>



Shader Development Environments 2/2

- Shdr online GLSL editor: <https://shdr.bkcore.com/>





universität
uulm

2.5 Fragment Tests and Operations

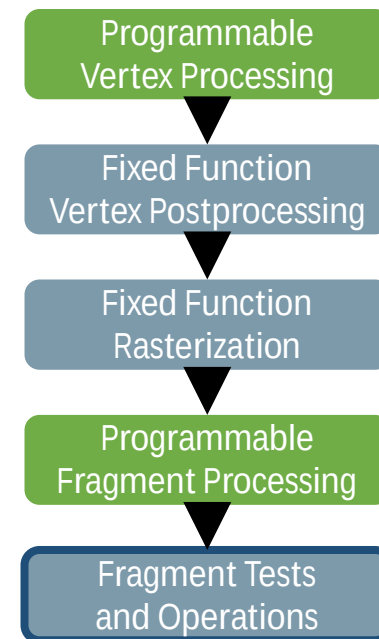
Towards the finishing line to the pixel



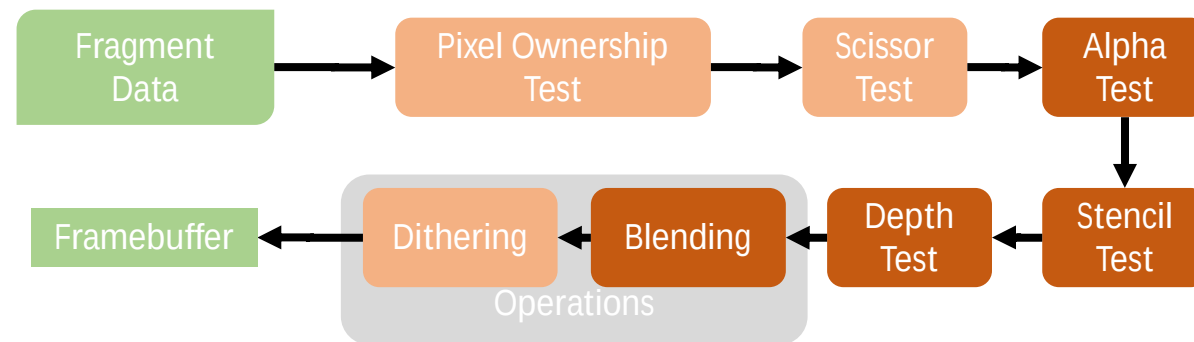
Visual Computing Group
Institute of Media Informatics

Fragment Tests and Operations 1/2

- After fragments have been generated, they must go through the per-fragment tests and operations
 - Only if all these tests are successful, a fragment will end up in the framebuffer and become a pixel



Fragment Tests and Operations 2/2



Alpha Test

- If framebuffer is an RGBA buffer, the alpha test can be performed
- The test must first be activated

```
glEnable(GL_ALPHA_TEST);
```

- The comparison function indicates which fragments survive the test

```
glAlphaFunc(func, value);
```

- **func** – accepts the following values
GL_NEVER, GL_LESS, GL_EQUAL, GL_GREATER, ...
- **value** – reference value to compare with

Stencil Test

- Stencil Buffer
 - Is a frame buffer component
 - Is (indirectly) filled by drawing operations
 - No special stencil drawing operations
- Stencil operation
 - Modification of the stencil buffer
- Stencil function
 - Execution with every drawing operation

- Stenciling is today mostly replaced with programmable shaders together with framebuffer objects

Depth Test

- If the framebuffer has a depth buffer, the depth test can be performed
- The test must first be activated

```
glEnable(GL_DEPTH_TEST);
```

- The comparison is made with the depth value already in the buffer
- The comparison function indicates which fragments survive the test

```
glDepthFunc(func);
```

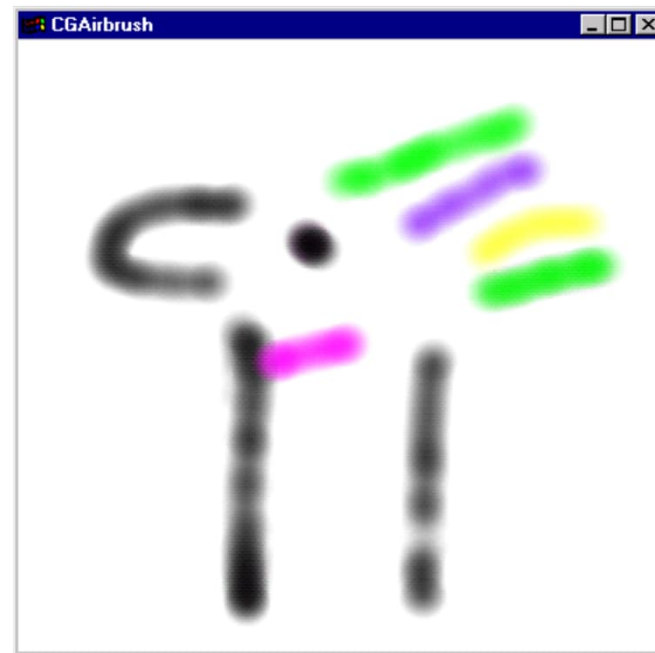
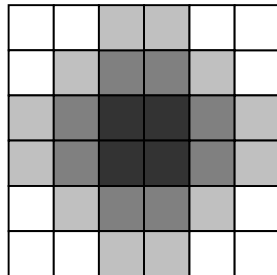
- **func** – accepts the following values
GL_NEVER, GL_LESS, GL_EQUAL, GL_GREATER, ...

Image Composition and Mixture

- Alpha values of the pixels
 - control the combination of pixel values
 - are specified as color components of color pixels
 - $P = [R, G, B, A]$, with $R, G, B, A \in [0,1]$
- Applications are manifold
 - Semi-transparent surfaces
 - Digital image combinations
 - Drawing and painting techniques
- Example: modeling transparency (alpha≈opacity)
 - $A = 0.0$ completely transparent
 - $A = 0.2$ 80% transparent (i.e., 20% opaque)
 - $A = 1.0$ 0% transparent (i.e., 100% opaque)

Blending Application

- Example application for transparency: drawing with semi-transparent brush



Blending Process 1/2

- Blending is an operation called per fragment
 - Fragments are modified after rasterization, but before writing to the framebuffer with a blending function
 - Combination of a source fragment to be written with the destination pixel in the frame buffer by using a blending function
- To use blending, two steps are applied
 - Determination of blending factors
 - Calculation of the pixel value from source fragment and destination pixel

Blending Process 2/2

- Source factor: F_s
- Destination factor: F_d
- Source fragment: $[P_{s,r}, P_{s,g}, P_{s,b}, P_{s,a}]$
- Destination pixel: $[P_{d,r}, P_{d,g}, P_{d,b}, P_{d,a}]$
- Blended pixel:
$$\begin{aligned} &[F_s \cdot P_{s,r} + F_d \cdot P_{d,r}, \\ &F_s \cdot P_{s,g} + F_d \cdot P_{d,g}, \\ &F_s \cdot P_{s,b} + F_d \cdot P_{d,b}, \\ &F_s \cdot P_{s,a} + F_d \cdot P_{d,a}] \end{aligned}$$

Applying Blending

- Switch on (or switch off) blending
 - `glEnable(GL_BLEND);`
 - `glDisable(GL_BLEND);`
- Defining Blending Factors
 - `glBlendFunc(srcFac, destFac);`
- Blending factors in OpenGL
 - `GL_ONE`
 - `GL_ZERO`
 - `GL_SRC_ALPHA`
 - `GL_ONE_MINUS_SRC_ALPHA`
 - `GL_DST_ALPHA`
 - `GL_ONE_MINUS_DST_ALPHA`
 - ...

Blending Examples

- Example 1: Copy source pixels and overwrite destination pixels
 - `glBlendFunc(GL_ONE, GL_ZERO);`
 - $P_{blend} = [P_{s,r}, P_{s,g}, P_{s,b}, P_{s,a}]$
- Example 2: Alpha-based mixing of source pixels and destination pixels
 - `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);`
 - $$P_{blend} = \begin{bmatrix} P_{s,a} \cdot P_{s,r} + (1 - P_{s,a}) \cdot P_{d,r}, \\ P_{s,a} \cdot P_{s,g} + (1 - P_{s,a}) \cdot P_{d,g}, \\ P_{s,a} \cdot P_{s,b} + (1 - P_{s,a}) \cdot P_{d,b}, \\ P_{s,a} \cdot P_{s,a} + (1 - P_{s,a}) \cdot P_{d,a} \end{bmatrix}$$



2.6 Framebuffer

Contains everything that is (in)visible on the screen

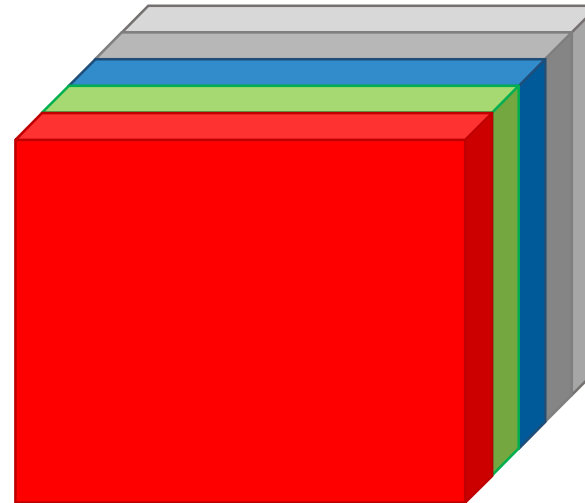


Viewport

- Rectangular area in pixel grid, set explicitly
- Rendering operations are limited to this area
- All primitives that are completely or partially outside are clipped at the viewport area (clipping)

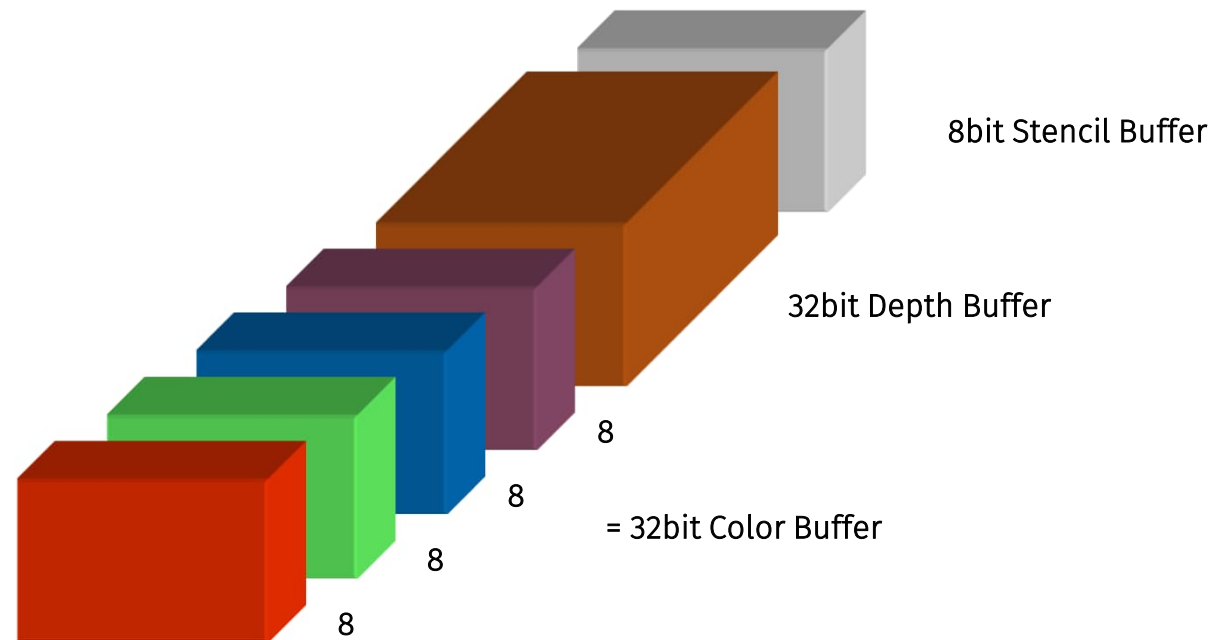
Framebuffer

- Context-bound raster with a resolution and structure specified by the application
- Physical realization on the GPU
- Logical subdivision
 - Color memory
 - Image mask memory (stencil buffer)
 - Depth buffer
 - ...



OpenGL Framebuffers 1/3

- Typical OpenGL framebuffer configuration



OpenGL Framebuffers 2/3

- All buffers of a frame buffer have the same grid size
- Number of bitplanes per buffer can be configured individually
- Individual buffers are directly supported by the hardware
- Buffer identifier
 - GL_COLOR_BUFFER_BIT
 - GL_DEPTH_BUFFER_BIT
 - GL_STENCIL_BUFFER_BIT
 - GL_ACCUM_BUFFER_BIT

OpenGL Framebuffers 3/3

- Delete a buffer

```
glClear(GLbitfield mask);
```

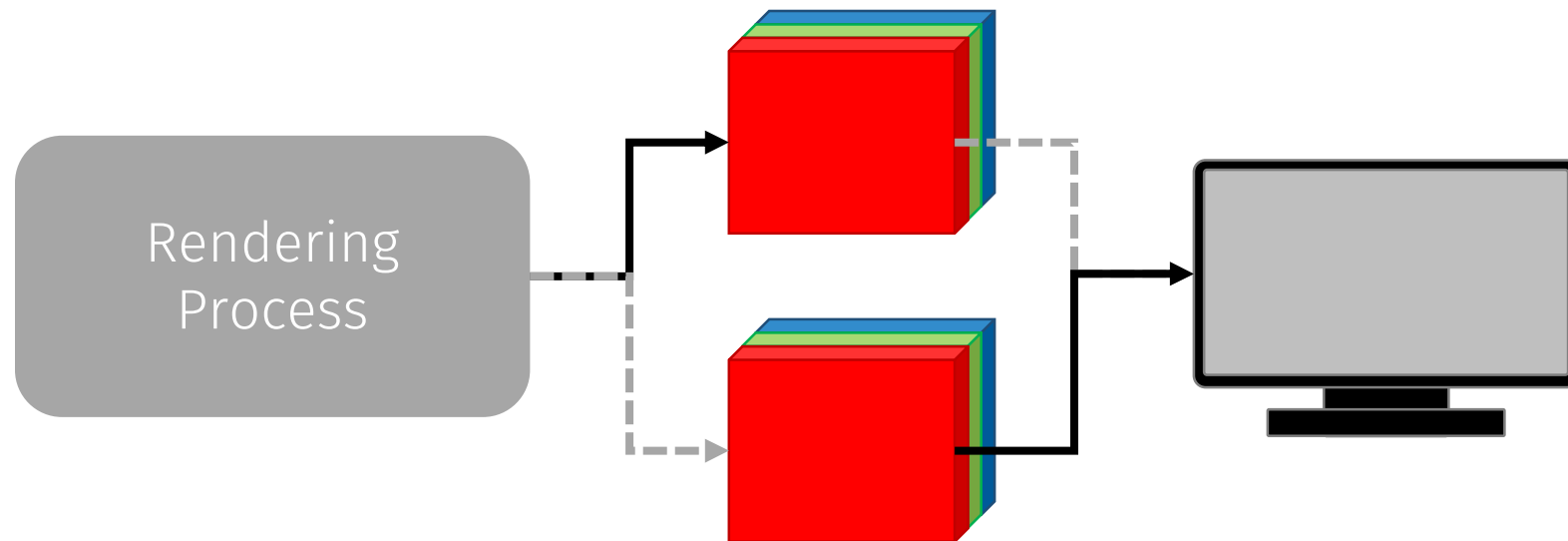
- Set the initialization color for the color buffer

```
glClearColor(1.0,1.0,1.0,1.0);
```

```
glClear(GL_COLOR_BUFFER_BIT);
```

Double Buffering

- Use of two color buffers to avoid flicker effects
 - Image buildup takes place in the background (back buffer)
 - Visible image is independent of this (front buffer)





universität
uulm

2.7 Pixel-based Rendering

Direct pixel processing without involving 3D models



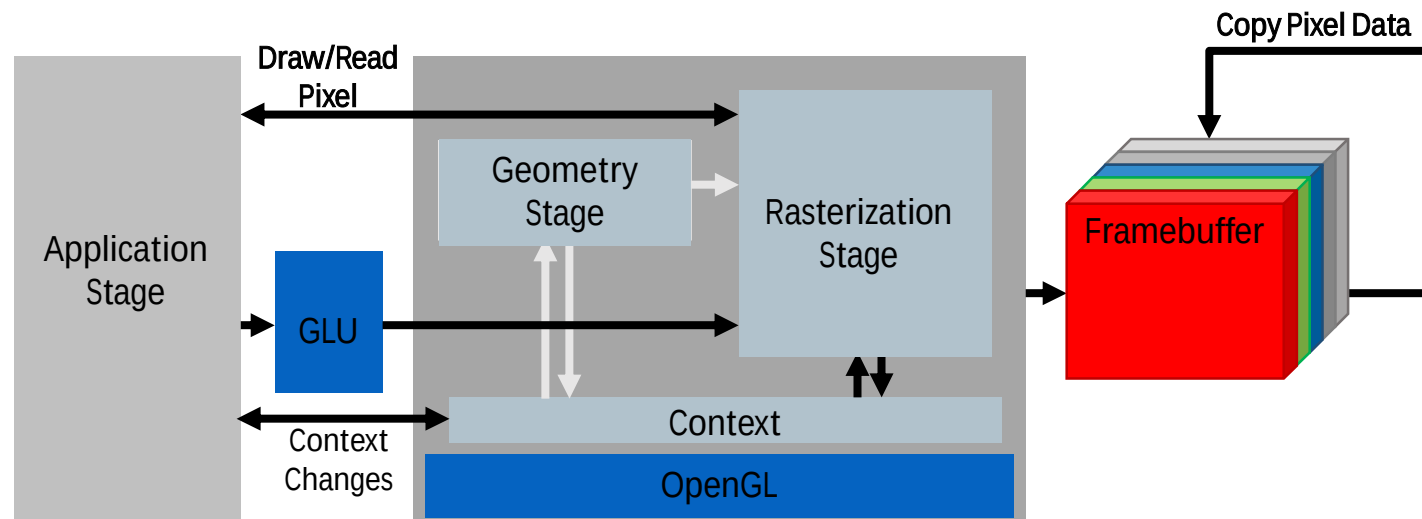
Visual Computing Group
Institute of Media Informatics

Pixel-based Rendering Operations

- Generally
 - Read and write operations for various image data formats, usually as an external library (e.g., DevIl)
 - Conversions between image data types
- In OpenGL done with framebuffer objects
 - `glDrawPixels()` – write pixel array from main memory to framebuffer
 - `glReadPixels()` – read pixel array from framebuffer to main memory (slow!)
 - `glCopyPixels()` – copy pixel array within the frame buffer

Image Operations

- Rendering image data with OpenGL



Write Pixels into Framebuffer

- `glDrawPixels(x, y, width, height, format, type, pixels);`
- Example

```
// 64x64 array with rgb components
GLubyte img[64][64][3];
void draw() {
    glClear(GL_COLOR_BUFFER_BIT);
    glRasterPos2i(0,0);
    glDrawPixels(16, 16,           // position
                64, 64,           // width and height
                GL_RGB,           // pixel format
                GL_UNSIGNED_BYTE, // pixel data type
                img                // pixel array to be copied
    );
    glFlush();
}
```

Read Pixels from Framebuffer

- `glReadPixels(x, y, width, height, format, type, pixels);`
- Example

```
GLubyte* img = 0;
int width, height;           // canvas size
void snapshot() {
    img = new GLubyte[width*height*3]; // allocation
    glReadPixels(0, 0,              // position
                 width, height,     // width and height
                 GL_RGB,             // pixel format
                 GL_UNSIGNED_BYTE,   // pixel data type
                 img                 // destination array);
}
```

Formats & Data Types

- Format information for pixel array operations
 - **GL_RGB** – red-green-blue values of the frame buffer (FB)
 - **GL_RED** – red values of the FB
 - **GL_ALPHA** – transparency values of the FB
 - **GL_RGBA** – red-Green-Blue-Alpha values of the FB
 - **GL_LUMINANCE** – brightness values of the FB
 - ...
- Data types for pixel array operations
 - **GL_UNSIGNED_BYTE** – a pixel component is represented as an 8-bit value
 - **GL_UNSIGNED_INT** – a pixel component is represented as a 32-bit integer value
- Example: `glReadPixels` with
 - Format = **GL_RGBA** and
 - Data type = **GL_UNSIGNED_INT**

=> 4 components (RGBA) per pixel are read in each one integer (4x4=16 bytes needed)

Chapter Summary

- Rendering pipeline consist of geometry stage and rasterization stage
- Geometric primitives (mostly triangles) are transformed through different coordinate systems in the geometry stage
- Rasterization results in fragments, which are pixel predecessors
- Fragments have to endure a sequence of tests in the rasterization stage, in order to be promoted to pixels



universität
uulm

References

Literature and other sources used in this chapter



Visual Computing Group
Institute of Media Informatics

References

- Text Books

- D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane: OpenGL Programming Guide: The Official Guide to Learning OpenGL (8th Edition), Addison-Wesley 2013.
- P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK Peters 2016. (Chapters 1, 3 & 21)